

Solution Architecture

Date	17 September 2026
Team ID	SWTID-2026-9627
Project Name	AI Fit Track API
Maximum Marks	4 Marks

Solution Architecture - Layered Component View

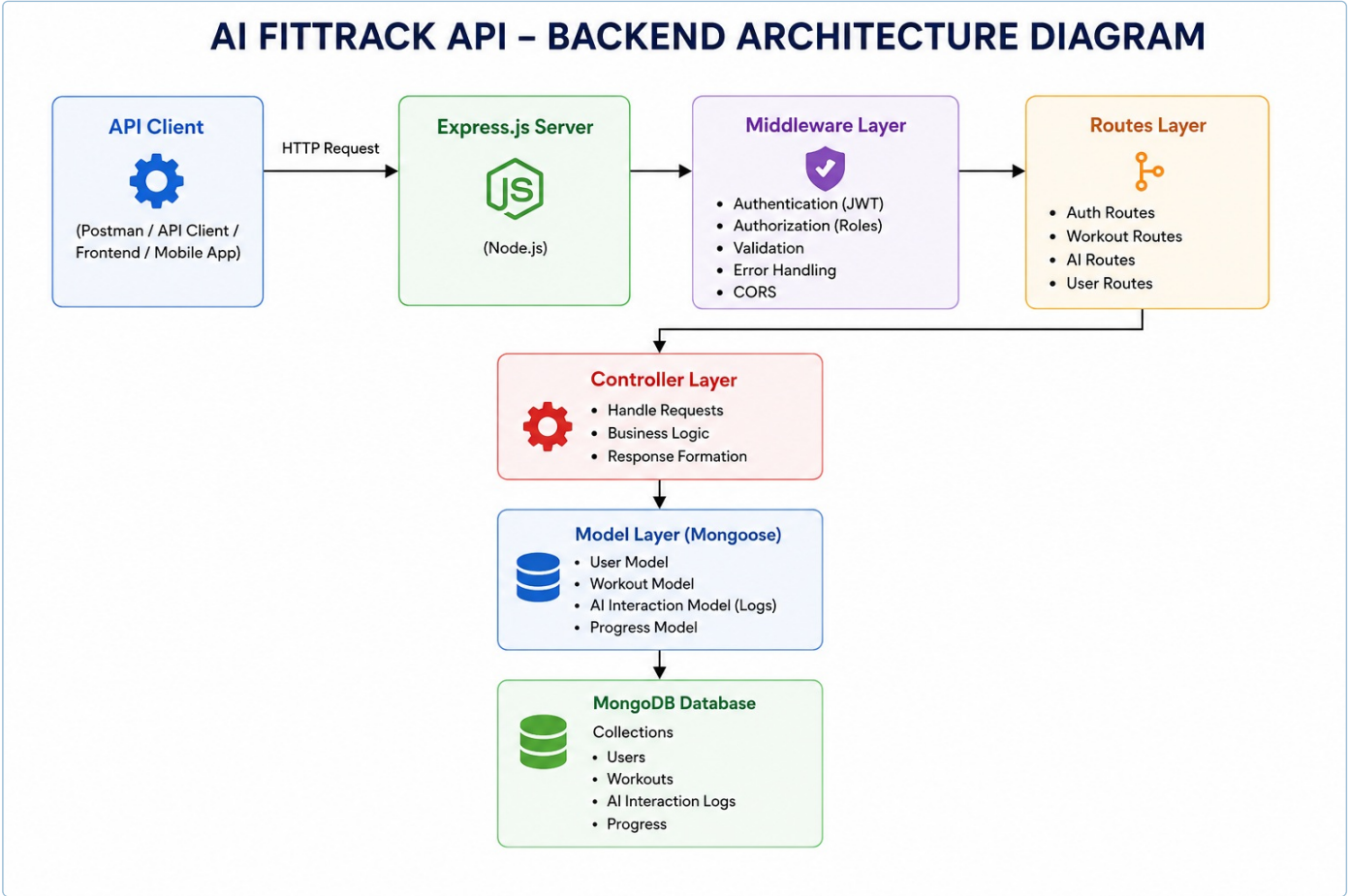


Fig. 1 – AI Fit Track layered architecture, as designed by the team

Component Description

Layer	Responsibility
Client Layer	Thunder Client / Postman during development and grading (a <code>client/</code> folder is reserved for a future React/mobile UI); sends HTTP requests with JSON bodies and, where required, a Bearer JWT.
Express Server / Gateway	<code>server.js</code> (or <code>app.js</code>) bootstraps Express, applies <code>cors</code> and JSON body-parsing middleware, and mounts all route groups.
Authentication Middleware	<code>middleware/auth.js</code> reads the <code>Authorization: Bearer <token></code> header, verifies the JWT with <code>jsonwebtoken</code> , and attaches the decoded user to <code>req.user</code> before allowing the request onward.
Route Layer	<code>routes/authRoutes.js</code> , <code>routes/workoutRoutes.js</code> , <code>routes/aiRoutes.js</code> declare each endpoint's path, HTTP method, and which middleware/controller handles it.
Controller Layer	<code>controllers/authController.js</code> , <code>workoutController.js</code> , <code>aiController.js</code> hold request-handling logic: input validation, calling models/services, and shaping the JSON response.
Service Layer	<code>services/geminiService.js</code> isolates all Google Gemini AI calls (prompt construction, response parsing) so controllers stay free of AI-vendor-specific code.
Database Layer	<code>models/User.js</code> and <code>models/Workout.js</code> (Mongoose schemas) plus <code>config/db.js</code> (MongoDB connection) persist and validate all data.

MVC Design Pattern

AI FITTRACK API (BACKEND) – MVC PATTERN

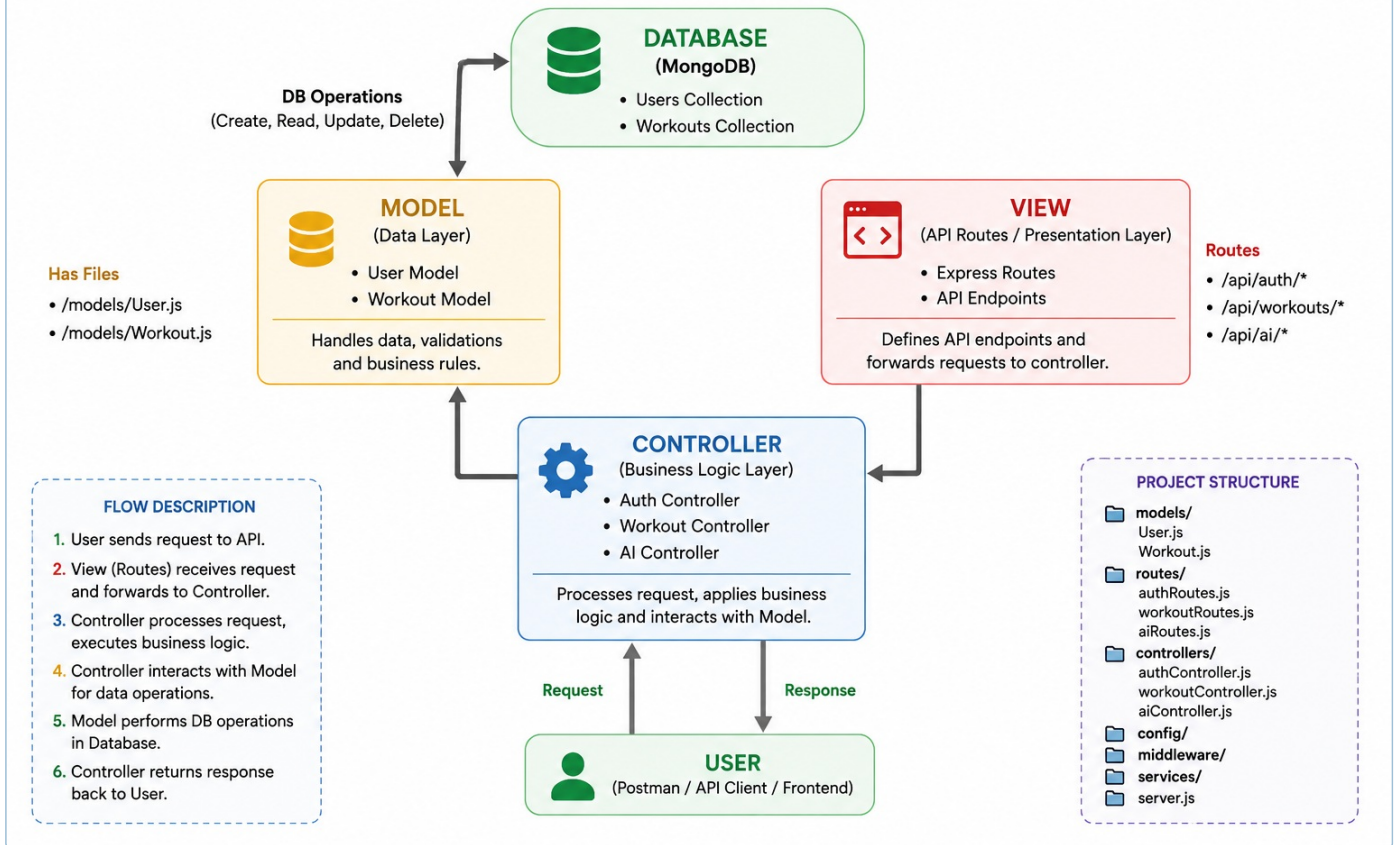


Fig. 2 – MVC mapping used across the codebase

Models (User , Workout) define schema and validation; Controllers contain the request/response logic and orchestrate calls to models and the Gemini service; there is no server-rendered View layer since the project is a pure JSON REST API – the "view" is the JSON response itself, consumed by whichever client (Thunder Client, Postman, or a future frontend) issues the request.